Vision-Guided Cobot Pick-and-Place Cell — Project Brief

Nov 9, 2026 · @Shabd

Overview

Build a small automated cell where a fixed camera finds parts on a tray, a cobot picks each one, and a vision check sorts it into a pass or reject bin. A PLC-style state machine runs the cell, with an operator HMI and full cycle logging, and a lightweight validation package proves it works.

This is the work Irish medtech machine builders (DesignPro, Modular Automation, Kaon), Irish Manufacturing Research and fab-heavy employers (Intel Leixlip, ADI Limerick) actually do: vision-guided handling, inspection and documented validation. It also complements Tequila, which already covers mobile navigation, by adding manipulation and industrial control.

Goals and success criteria

The cell is done when it hits every target below over a 200-part run. The numbers are starting targets: tighten or loosen them once you know the cobot and gripper.

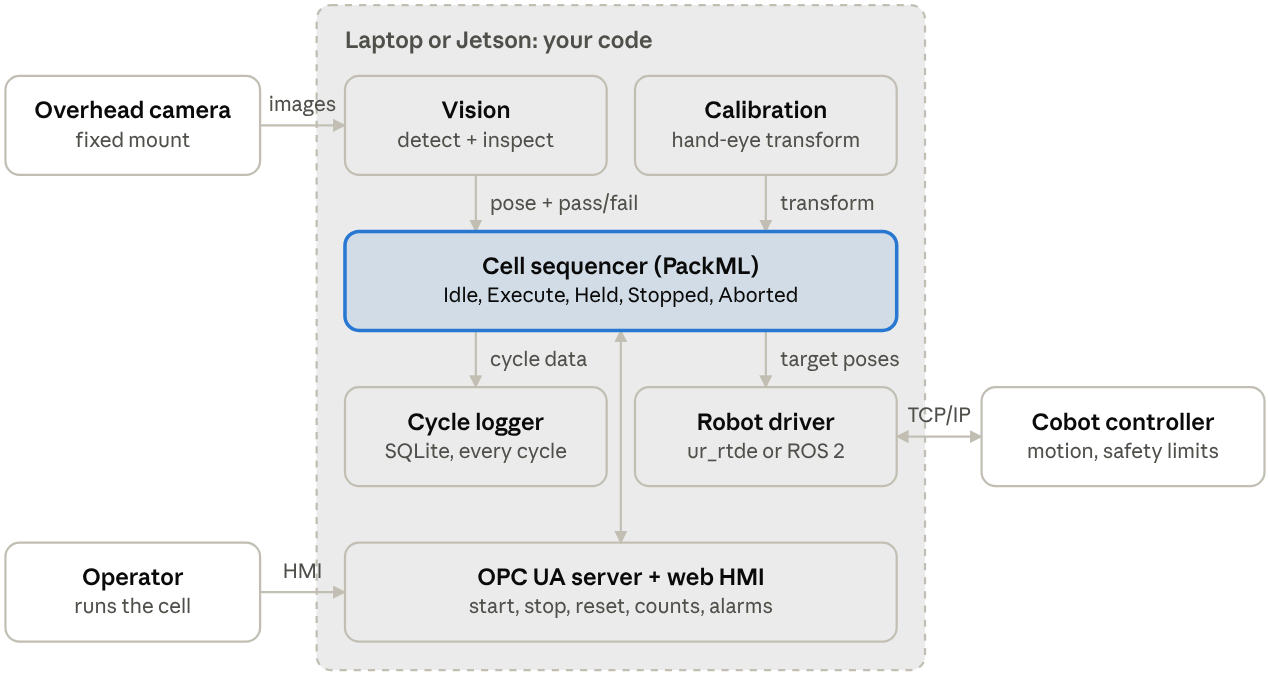
Metric	Target	How it's measured
Pick success rate	⩾ 95%	Successful picks ÷ attempts over 200 cycles
Placement accuracy	± 2 mm	Part position in the drop fixture vs nominal, 20 samples
Defect detection (recall)	⩾ 95%	Seeded-defect parts correctly rejected
False reject rate	⩽ 5%	Good parts wrongly rejected
Cycle time	⩽ 10 s per part	Timestamp from detection to place, at the safety-limited speed
Fault recovery	100%	Cell resumes from HELD/STOPPED after e-stop or a failed pick, with no restart

Metric	Target	How it's measured
Traceability	100%	Every cycle logged with image, result, pose and timestamp

System architecture

The sequencer is the only component that commands the robot. Vision and calibration feed it part poses and pass/fail results. The OPC UA layer exposes its state to the HMI, and the cobot controller still enforces every safety limit.

Your code runs the cell; the cobot only moves and enforces safety



cell architecture · 8 components, 1 control loop

One cycle: grab image → detect part and estimate (x, y, θ) → transform to robot base frame → inspect → pick → place in the pass or reject bin → log → repeat.

Hardware and software stack

Everything except motion and safety runs on your machine, and the cobot's firmware stays untouched. Picks are planar (x, y, θ on a flat tray), which keeps the vision and calibration manageable.

Layer	Choice	Notes
Robot	College cobot (model TBD)	Controlled through its external API or ROS 2 driver, e.g. External Control URCap + `ur_rtde` on a UR
Gripper	Whatever the cobot has (vacuum or 2-finger)	Vacuum is more forgiving of pose error
Camera	Fixed overhead USB camera, or RealSense D435 / OAK-D if available	Eye-to-hand setup. Diffuse lighting matters more than resolution
Compute	Laptop with GPU, or a Jetson	Runs vision, sequencer, HMI, logging
Parts	3D-printed medtech-style parts (e.g. luer-connector lookalikes) or M4–M8 fasteners	Print a batch with seeded defects: short shots, missing features, flash
Calibration	ChArUco board + OpenCV `calibrateCamera` and `calibrateHandEye`	Maps pixels to robot base frame
Detection and pose	YOLO (v8 or newer) with oriented boxes, or classical contours + `minAreaRect` to start	Start classical. Switch to learned once it works end to end
Inspection	Anomaly detection trained on good parts only (PatchCore via `anomalib`)	Matches how real lines work: defects are rare and varied
Motion	ROS 2 (Humble or Jazzy) + MoveIt 2, or direct API calls	Direct API is faster to get running. ROS 2 looks better on a CV
Sequencer	Python state machine following PackML states (Idle, Starting, Execute, Held, Stopped, Aborted)	The PackML vocabulary is what industry engineers recognise
Interface	OPC UA server (`asyncua`) exposing cell tags, plus a small web HMI	This is the SCADA piece: start/stop/reset, counts, last image, alarms

Layer	Choice	Notes
Logging	SQLite (or InfluxDB + Grafana)	One row per cycle: timestamp, part ID, pose, result, anomaly score, cycle time, image path

Milestones

About 11 weeks part-time alongside coursework. Each phase ends with something that runs, so you always have a demo even if you stop early.

- [ ] **Week 1: Access and connection.** Confirm the cobot model, get lab permission, install the external-control plugin, and send a single move-to-pose from your laptop.
- [ ] **Weeks 2–3: Vision on the bench.** Mount the camera, fix the lighting, and detect part position and orientation on the tray at ≥ 98% detection.
- [ ] **Weeks 4–5: Calibration and first picks.** Run hand-eye calibration, convert detections to robot-frame poses, and complete open-loop pick-and-place of one part type.
- [ ] **Weeks 6–7: Sequencer, HMI and logging.** Add the PackML state machine, the OPC UA tags and web HMI, the per-cycle log and fault handling.
- [ ] **Weeks 8–9: Inspection.** Collect good-part images, train the anomaly model, add the pass/reject decision and bin routing, and tune the threshold.
- [ ] **Weeks 10–11: Validation and write-up.** Run the test protocol and the 200-cycle run, then write the report, README and demo video.

Validation package

A short IQ/OQ/PQ-style protocol is what will set this apart: few student projects have one, and in medtech every machine ships with one. Write the acceptance criteria before you run the tests, then record pass or fail against them.

Test ID	Stage	What it checks	Acceptance criterion
IQ-01	Installation	Software versions, robot safety config and calibration file recorded	Versions and calibration file hash match the build record

Test ID	Stage	What it checks	Acceptance criterion
IQ-02	Installation	Camera mount and lighting fixed and documented	Photo and dimensions in the report
OQ-01	Operational	Detection across 0–360° part orientations	≥ 98% detected, angle error ≤ 3°
OQ-02	Operational	Detection under lighting variation (±30% brightness)	≥ 95% detected
OQ-03	Operational	E-stop and protective stop mid-cycle	Cell goes to STOPPED or HELD and resumes without restarting software
OQ-04	Operational	Failed pick (part removed before grasp)	Fault raised, logged and recovered automatically
OQ-05	Operational	Seeded-defect set (≥ 30 defective parts)	Recall ≥ 95%
PQ-01	Performance	200 consecutive cycles, mixed good and defective parts	Every target in Goals met, every cycle logged

The deliverable is a 4–6 page report: scope, setup, the protocol above, results, deviations and what you'd change.

Safety and access rules

The robot's own safety configuration is the final safeguard, so it never gets switched off. Agree these rules with whoever runs the lab before week 1.

- Get written OK from the lab supervisor to install the external-control plugin and to use remote mode.
- Keep the controller's speed, force and joint limits and any safety planes enabled. Develop at reduced speed (≤ 25%).
- Keep the e-stop within reach whenever the robot is under software control. Never leave it running unattended.
- Before running, write a one-page risk assessment for the cell layout, referencing ISO 10218 and ISO/TS 15066 for collaborative operation.

- Test every new motion in simulation or at minimum speed first. Clamp all software-generated targets to a defined workspace box before sending them.
- Keep the robot network isolated from the college network unless IT says otherwise.

Deliverables and portfolio framing

Ship four things, and lead with the numbers from PQ-01.

1. **GitHub repo**: clean modules (vision, calibration, sequencer, HMI, logging), a README with the architecture diagram, and setup steps someone else could follow.
2. **Validation report (PDF)**: the IQ/OQ/PQ protocol and results from the section above.
3. **2-minute demo video**: normal run, a defect being rejected, an e-stop and recovery, and the HMI on screen.
4. **Short write-up** (LinkedIn or blog): the problem, what you built, the results and one thing that went wrong.

CV line to aim for: *"Built a vision-guided cobot pick-and-inspect cell (ROS 2, OpenCV, anomaly detection, OPC UA). Validated with IQ/OQ/PQ protocol: 96% pick rate, 97% defect recall, 8.4 s cycle over 200 parts."* Swap in your real numbers.

How to pitch it to each audience:

- **Machine builders (DesignPro, Modular, Kaon, Greatech)**: validation, PackML, fault recovery, cycle time.
- **IMR and Digital Manufacturing Ireland**: vision-guided robotics, anomaly detection and possible digital-twin extensions.
- **Intel, ADI, Valeo**: the perception pipeline, calibration accuracy and data logging.

Open questions and stretch goals

The cobot model decides the driver and the gripper, so settle the first two questions before week 1.

- [ ] Which cobot brand and model is it, and which controller software version?
- [ ] Will the lab allow the external-control plugin and remote mode?
- [ ] What gripper is fitted (vacuum or fingers), and what payload and stroke does it have?
- [ ] Is a depth camera available, or only a USB webcam?
- [ ] How many hours a week can you book on the robot?

Stretch goals, once the core cell passes PQ-01:

- **Learned representations for inspection**: swap PatchCore's backbone for your own JEPA-style encoder and compare recall. This connects to Ask the World.
- **Write the sequencer in Rung**: compile the PackML state machine from your own control language. It becomes a real test case for the compiler.
- **Bin picking with depth**: move from a flat tray to jumbled parts using 6-DoF pose from the depth camera.
- **Digital twin**: mirror the cell in Isaac Sim or Gazebo, driven by the same OPC UA tags.